Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera: Software

Análisis crítico de un sistema distribuido: HDFS comparado con Ceph

Informe académico individual (Unidad 1)

Asignatura:	Sistemas Distribuidos
Código de la actividad:	TA-IND-01
Estudiante:	Silva Triviño John Jairo
Semestre y paralelo:	Séptimo semestre, paralelo A
Docente:	Guerrero Ulloa Gleiston Cicerón
Grupo:	TA-GHLZ
Fecha:	13 de noviembre de 2026

Repositorio:

`https://github.com/ameraj2-bit/ta-ind-01-sistemas-distribuidos`

Resumen

Este informe analiza el Hadoop Distributed File System (HDFS) como sistema distribuido real y lo compara con Ceph. Describo su arquitectura, con un NameNode que guarda los metadatos y varios DataNodes que guardan bloques replicados, y evalúo qué transparencias ofrece, cómo se ubica frente a los teoremas CAP y PACELC, qué modelos de comunicación usa y cómo tolera fallos, incluido un incidente real documentado en LinkedIn. Para comprobarlo monté un clúster propio con Docker, de un NameNode y cuatro DataNodes, y apagué uno mientras leía un archivo de 300 MB. El NameNode tardó 14 min 02 s en declarar muerto al nodo; aun así el archivo siguió disponible y sus bloques volvieron a tener tres réplicas. Contrasto estos resultados con dos situaciones de mi entorno, el SGA de la UTEQ y WhatsApp, y con Ceph, que ubica los datos mediante el algoritmo CRUSH y no mediante un servidor central. Concluyo que HDFS es adecuado para archivos grandes que se escriben una vez, pero que su NameNode único concentra el riesgo y limita el crecimiento.

1. Introducción

Un sistema distribuido reparte datos y trabajo entre varias máquinas que se coordinan mediante mensajes, y debe seguir funcionando aunque alguna falle. HDFS resuelve el problema de almacenar archivos muy grandes en máquinas comunes: los divide en bloques y los copia en distintas máquinas. Es un marco ampliamente usado para el procesamiento de grandes volúmenes de datos [1]. A escala real, LinkedIn reporta 1 exabyte de datos en el conjunto de sus clústeres Hadoop, con un clúster de 10 000 nodos que almacena 500 PB y un solo NameNode que mantiene unos 1000 millones de objetos entre directorios, archivos y bloques [2]. Elegí HDFS porque es un caso claro de ese problema y porque puedo reproducirlo a pequeña escala en mi computador. Como segundo sistema para comparar elegí Ceph, que resuelve el mismo problema de otra forma: en lugar de un servidor que recuerda dónde está cada dato, calcula la ubicación [3].

El objetivo es entender y evaluar de forma crítica cómo HDFS responde a las propiedades de la Unidad 1: transparencia, consistencia y disponibilidad, comunicación y tolerancia a fallos. El trabajo combina tres fuentes de evidencia: documentación y artículos citados, un experimento propio de caída de un DataNode y situaciones reales de mi entorno. El informe sigue este orden: arquitectura (sección 2), análisis (sección 3), experimento (sección 4), ejemplos del entorno (sección 5), comparación con Ceph (sección 6), postura crítica y conclusiones.

2. Arquitectura de HDFS

HDFS sigue un esquema maestro y trabajadores. El **NameNode** es el árbitro y el repositorio de todos los metadatos de HDFS [4]: el árbol de directorios y, para cada archivo, la lista de bloques y los DataNodes donde está cada uno. Los **DataNodes** guardan los bloques; un tamaño típico de bloque es de 128 MB [4]. Cada DataNode envía periódicamente un latido (*heartbeat*) al NameNode, y el NameNode detecta un problema por la ausencia de ese latido [4]. Los archivos son de escritura única (salvo anexos y truncados) y tienen un solo escritor a la vez [4].

La figura 1 muestra el clúster que armé para el experimento. Los tres bloques del archivo de prueba aparecen con la ubicación real que devolvió el comando `hdfs fsck` (sección 4).

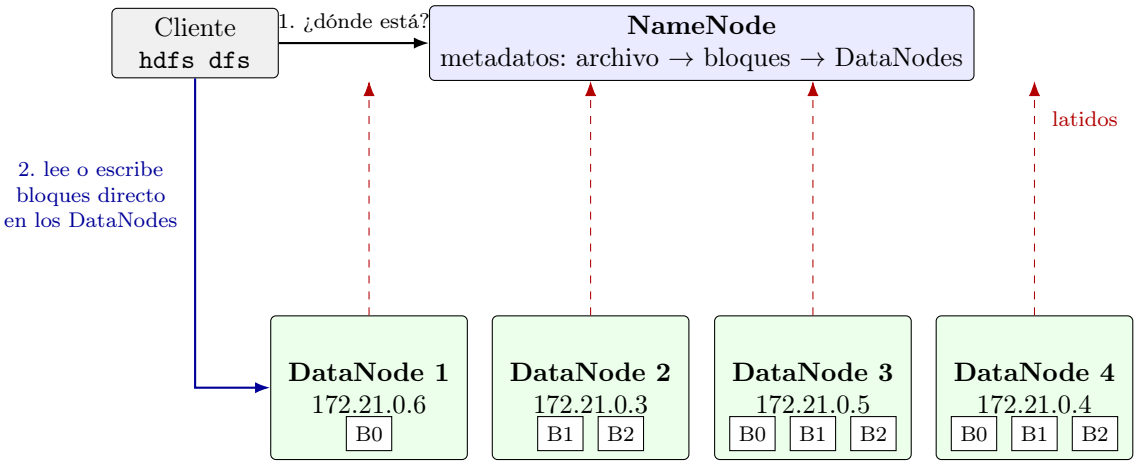


Figura 1: Clúster del experimento: 1 NameNode y 4 DataNodes, replicación 3. B0, B1 y B2 son los tres bloques de `/datos.bin` (300 MB); su ubicación proviene de mi `fsck` (captura-05). Línea discontinua: latidos. Línea azul: lectura o escritura directa de bloques (en la escritura los DataNodes se encadenan en una tubería). Diagrama propio dibujado en TikZ a partir de mi boceto a mano.

2.1. Evolución desde mi boceto a mano

Mi primer esquema (Anexo A, fechado 12/11/2026) mostraba un NameNode, tres DataNodes y un cliente, con los pasos «1. ¿Dónde está el archivo?» y «2. Lee los bloques directo de un DataNode», los latidos cada pocos segundos, una tubería de escritura DN1 → DN2 → DN3 y los racks como elemento opcional. Los cambios al diagrama definitivo fueron:

- **Cuatro DataNodes en lugar de tres.** Con tres nodos y factor de replicación 3, todos los bloques están en todos los nodos y, si uno cae, no queda un nodo libre donde recrear la réplica. Con cuatro hay un nodo disponible para ello.
- **Ubicación real de los bloques.** En el boceto cada DataNode tenía los bloques A y B. En el clúster real el archivo se dividió en tres bloques (128 MB, 128 MB y 44 MB) y la ubicación no es uniforme: el DataNode 1 solo tiene B0.
- **Datos reales del montaje:** direcciones IP, un solo rack, y el NameNode atendiendo solicitudes en el puerto 8020 y la interfaz web en el 9870.

3. Análisis desde los conceptos de la Unidad 1

3.1. Transparencias de distribución

Transparencia	¿La ofrece?	Por qué, con base en mi experimento y las fuentes
Acceso	Sí	Leí `/datos.bin` con el mismo comando (`hdfs dfs`) sin importar en qué nodo estaba cada bloque.
Ubicación	Sí	Uso la ruta del archivo; el NameNode resuelve en qué DataNodes está cada bloque [4].
Migración	No la probé	No moví datos de un nodo a otro por decisión mía. Lo más cercano que observé fue la re-replicación tras la falla, que no es una migración pedida.
Reubicación	No la probé	No moví un nodo ni un bloque mientras el archivo estaba en uso.
Replicación	Sí	Veo un solo archivo; HDFS mantiene 3 copias de cada bloque (factor 3 en mi montaje).
Concurrencia	Limitada	HDFS admite un solo escritor por archivo [4], así que dos clientes no pueden escribir a la vez el mismo archivo.
Fallos	Parcial	Con `datanode4` apagado la lectura completa funcionó con el mismo comando y la misma ruta, pero el administrador sí ve nodos muertos en los informes y la detección tardó 14 min 02 s.

Cuadro 1: Transparencias de distribución en HDFS según mi montaje.

3.2. CAP y PACELC

El teorema CAP establece un compromiso entre consistencia, disponibilidad y tolerancia a particiones: un sistema solo puede conservar dos de las tres. PACELC lo extiende: si hay partición (P), se elige entre disponibilidad (A) y consistencia (C); si no la hay (E), se elige entre latencia (L) y consistencia (C) [5]. El artículo agrupa los sistemas en PC/EC, PC/EL, PA/EC y PA/EL, y en su lista de la página 172 figuran HBase y BigTable como PC/EC; HDFS no aparece clasificado.

Mi clasificación de HDFS (razonamiento propio, no una clasificación publicada): lo clasifico como PC/EC, es decir, del lado de la consistencia tanto ante una partición como sin ella. El NameNode es el árbitro único de los metadatos y el archivo tiene un solo escritor [4]; en mi montaje, que no tiene NameNode de respaldo, una falla del NameNode dejaría sin servicio al espacio de nombres antes que permitir vistas distintas del sistema de archivos (prioriza consistencia frente a disponibilidad). Sin partición, toda consulta de ubicación pasa por ese árbitro único y hay un solo escritor por archivo, lo que prefiere consistencia frente a latencia. No probé esa caída. Mi experimento fue una falla de un DataNode, no una partición de red, así que

no sirve como evidencia de PACELC: solo muestra que los datos siguieron disponibles gracias a la replicación.

Para Ceph, el artículo de Peters y van der Ster señala que CephFS busca ofrecer garantías de consistencia equivalentes a las de un sistema de archivos local [6], y la documentación indica que los monitores usan el algoritmo Paxos y una mayoría para acordar el mapa del clúster [3]. Esto es coherente con una orientación a consistencia, pero tampoco es una clasificación publicada.

3.3. Modelos de comunicación

HDFS combina tres formas de comunicación:

- **Cliente–servidor con llamada a procedimiento remoto** entre el cliente y el NameNode para preguntar ubicaciones (en mi configuración, `namenode:8020`). Es síncrona: el cliente espera la respuesta.
- **Transferencia directa de datos** entre el cliente y los DataNodes (puerto 9866 en las ubicaciones que mostró `fsck`), sin pasar por el NameNode. Esto evita que el NameNode sea un cuello de botella para los datos. Un estudio sobre archivos pequeños en HDFS describe el mismo flujo de lectura: el cliente pide al NameNode los metadatos (bloques y ubicaciones) y luego contacta a los DataNodes [7].
- **Mensajes periódicos (latidos)** de los DataNodes al NameNode [4], de tipo asíncrono y unidireccional. Su ausencia es la señal de falla.

En la escritura, los DataNodes que guardan las réplicas se encadenan en una tubería [4], como dibujé en mi boceto.

Clasificación. Es un modelo cliente–servidor en su variante maestro y trabajadores, no un modelo entre pares: el cliente siempre empieza preguntando al NameNode. Solo la tubería de escritura tiene una parte entre pares (DataNode a DataNode), por lo que lo considero híbrido. Las consultas al NameNode son síncronas por llamada remota; los latidos son asíncronos; la transferencia de bloques es un flujo directo de datos. En mi montaje no observé ningún bróker de mensajería con colas.

3.4. Tolerancia a fallos

Los mecanismos son tres: replicación de bloques, detección por latidos y re-replicación. Ante la ausencia de latidos, el NameNode lleva la cuenta de qué bloques necesitan replicarse e inicia la replicación cuando hace falta [4]. Una revisión sobre tolerancia a fallos en sistemas de almacenamiento masivo describe el enfoque de latidos, usado en HDFS, YARN y Storm, y explica que un componente se declara fallido si no recibe el latido dentro de un tiempo especificado [8].

Qué fallos soporta y cuáles no.

- *Probado:* la caída de un DataNode, con replicación 3 en un clúster de 4 nodos.
- *Por la definición de la replicación (no probado):* con 3 copias por bloque, un bloque sigue existiendo mientras sobreviva al menos una de ellas.
- *No soportado en mi montaje:* la caída del NameNode, porque tengo uno solo y sin respaldo. No la probé. Esto también significa que mi montaje no usa consenso entre varios coordinadores.
- *Reintentos:* no los probé; mis lecturas funcionaron en el primer intento.
- *Fallos que afectan a muchos nodos a la vez:* el incidente que sigue muestra que la replicación no basta.

Incidente real documentado. En 2020 LinkedIn publicó el análisis de un incidente en su clúster Hadoop: por fallas de procedimiento en la gestión del ciclo de vida de los equipos, aproximadamente el 2 % de las máquinas de unos pocos racks fue reinstalado por error [9]. La pérdida fue pequeña en bloques (cerca del 0,02 %), pero el 10 % de sus flujos de trabajo falló porque basta un archivo corrupto para detener un flujo, y la recuperación tomó varios días [9]. Lo relevante para este informe es que la replicación no protege frente a errores que afecten a muchos nodos a la vez, y que la recuperación acelerada también estresa el sistema. En mi experimento fallaba un solo nodo, el caso para el que HDFS está diseñado.

Límite del NameNode. Un artículo sobre federación indica que HDFS escala los DataNodes pero no el NameNode, donde se gestionan los metadatos, lo que genera problemas de rendimiento y de crecimiento del espacio de nombres [1]. En LinkedIn, el NameNode del mayor clúster necesita un montículo de memoria de 380 GB para mantener unos 1100 millones de objetos [2], un ejemplo concreto de que el conocimiento centralizado consume memoria a medida que crece el sistema.

4. Experimentación propia

Montaje. `docker compose` con la imagen `apache/hadoop:3`: 1 NameNode y 4 DataNodes, factor de replicación 3. Los archivos `docker-compose.yml` y `config`, junto con todos los comandos y salidas, están en la carpeta `experimento/` del repositorio. Subí un archivo de 300 MB (`/datos.bin`), que HDFS dividió en 3 bloques, cada uno con 3 réplicas.

Procedimiento. Apagué `datanode4` con `docker stop` a las 23:28:51 del 12/11/2026 y monitoreé el estado cada 60 s. Luego leí el archivo completo, revisé la ubicación de los bloques con `fsck` y volví a encender el nodo.

Hora local	Observación
23:24:35	`fsck`: 3 bloques, cada uno con 3 réplicas, 4 DataNodes (captura-05)
23:28:51	`docker stop datanode4`
23:31:12	El NameNode aún muestra 4 DataNodes vivos
23:42:53	Log del NameNode: *lost heartbeat from 172.21.0.4* (captura-10)
23:52:47	1 DataNode muerto, 3 vivos, 0 bloques subreplicados (captura-08)
00:03:15	`fsck`: 3 DataNodes, bloques con 3 réplicas, sin 172.21.0.4 (captura-09)
00:04:21	Lectura completa con el nodo caído: 1,853 s (captura-09)
00:10:42	`docker start datanode4`
00:11:10	4 DataNodes vivos, 0 bloques subreplicados (captura-11)

Cuadro 2: Línea de tiempo del experimento (horas locales, UTC−5).

Resultados.

- **Detección:** el NameNode declaró muerto al nodo 14 min 02 s después de apagarlo (23:28:51 → 04:42:53 UTC en el log). La detección no es inmediata: la documentación habla de ausencia de latidos [4] y la revisión de Saadoon et al. de un tiempo especificado [8]. No verifiqué con `hdfs getconf` los valores de latido configurados en mi clúster, así que no puedo explicar con certeza por qué fueron 14 min 02 s y no otro valor.
- **Disponibilidad y transparencia:** la lectura completa con 3 DataNodes vivos funcionó sin cambiar el comando, en 1,853 s; con 4 DataNodes tardó 1,690, 1,901 y 2,107 s (promedio 1,899 s). La diferencia entre 3 y 4 nodos (0,046 s) es menor que la variación entre lecturas con 4 nodos (0,417 s), así que no permite concluir nada sobre rendimiento.
- **Re-replicación:** antes de la falla el `datanode4` tenía una copia de cada bloque; a las 00:03:15 los bloques 1 y 2 estaban en los tres nodos restantes (172.21.0.3, .5 y .6) y el resumen de `fsck` indicaba replicación promedio 3,0 y 0 bloques subreplicados. No observé el proceso de copia, solo su resultado.

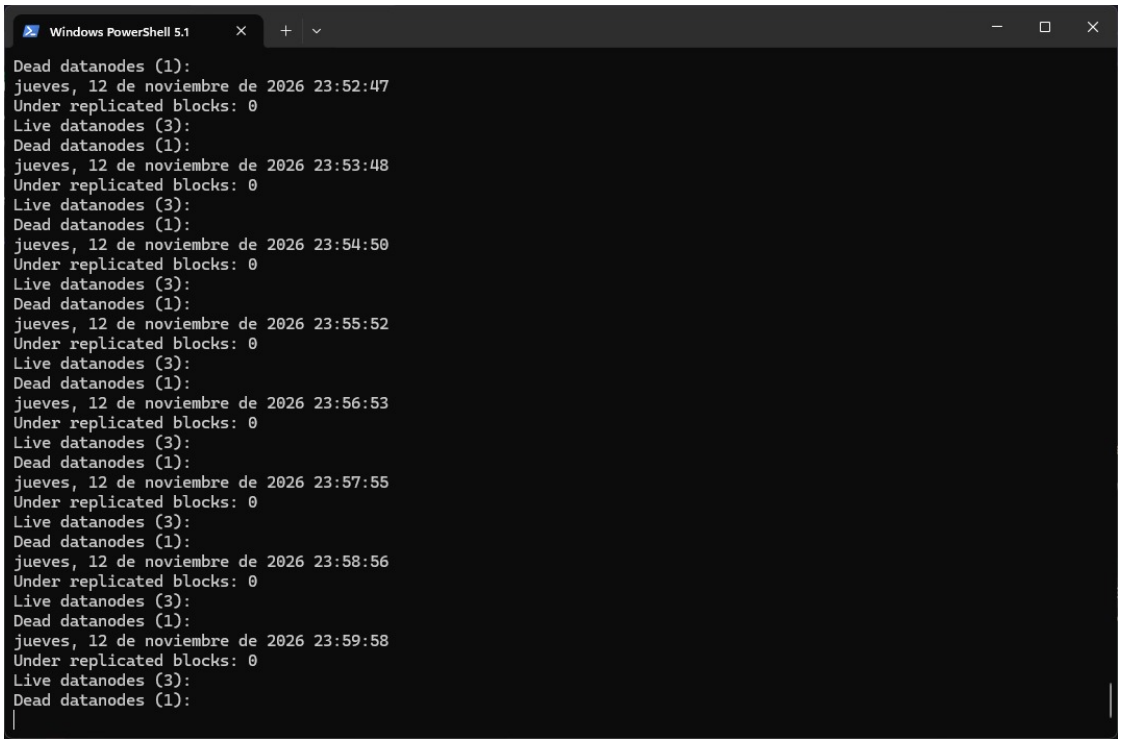


Figura 2: Captura propia (23:52): `Dead datanodes (1)`, `Live datanodes (3)` y 0 bloques subreplicados.

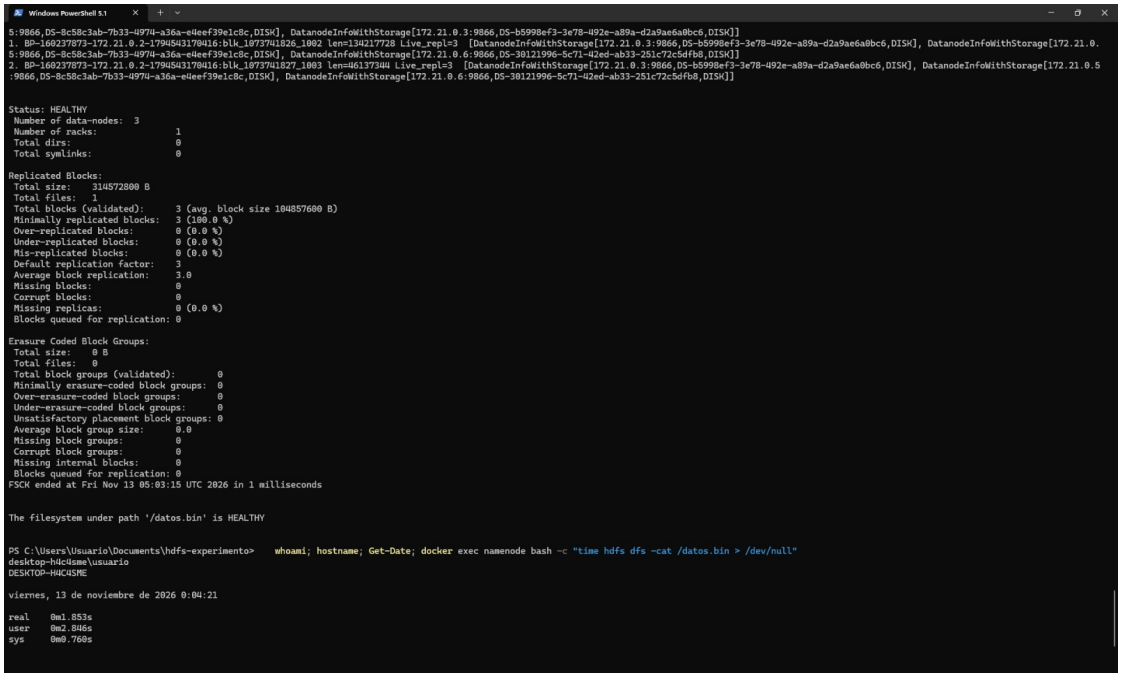


Figura 3: Captura propia (00:03–00:04): `fsck` con 3 DataNodes y lectura completa con el nodo caído.

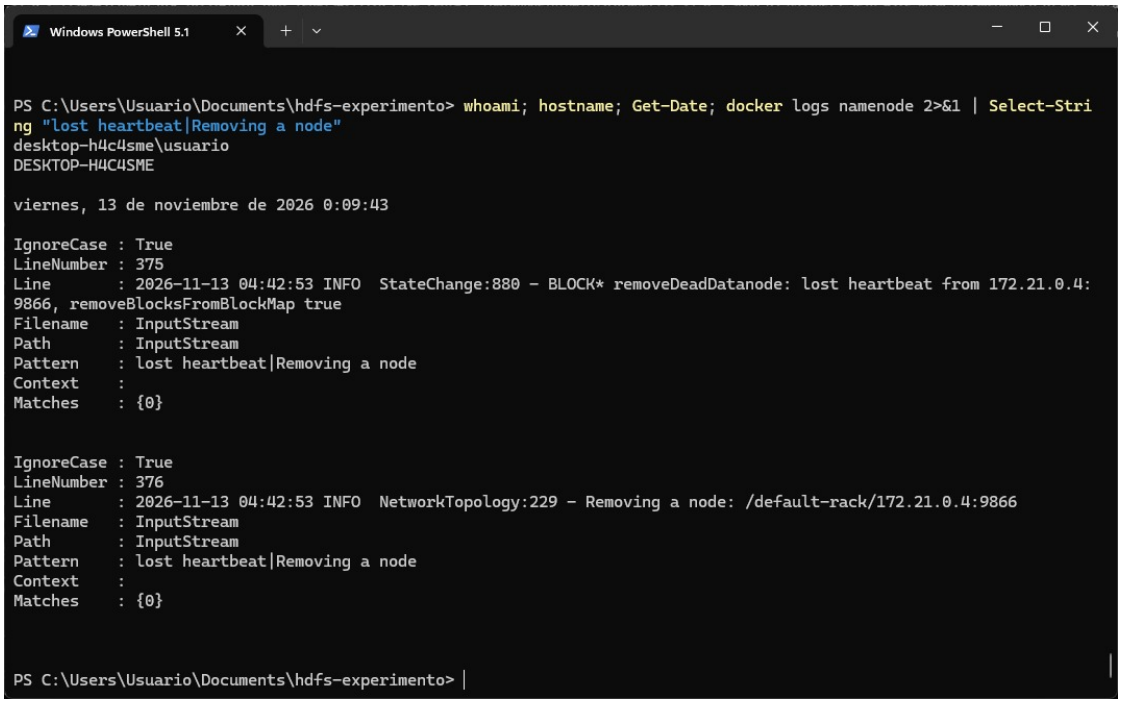


Figura 4: Captura propia: mensaje del NameNode *lost heartbeat from 172.21.0.4:9866*.

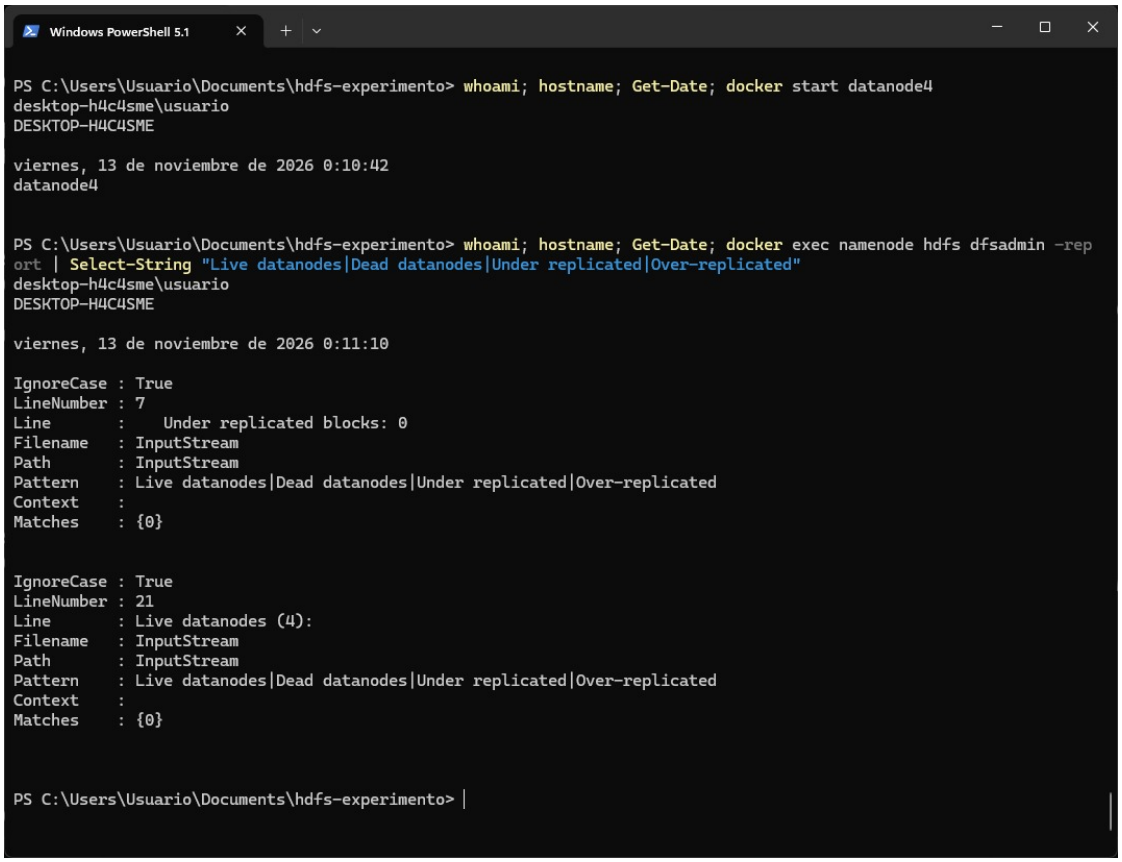


Figura 5: Captura propia (00:11): nodo recuperado, 4 DataNodes vivos y 0 bloques subreplicados.

Limitaciones. Los cuatro DataNodes comparten el disco de mi computador, así que simulé la caída de un contenedor y no una falla independiente de almacenamiento. Monitoreé cada 60

s, por lo que no capté el instante de la re-replicación. Hice una sola repetición y no medí la lectura antes de la falla ni durante los minutos en que el nodo estaba apagado pero aún no declarado muerto. No probé fallos simultáneos ni la caída del NameNode. El detalle completo está en `experimento/resultados.md`.

5. Ejemplos de mi entorno

5.1. SGA de la UTEQ: lentitud en un período de actividad académica

Aproximadamente en octubre de 2026, durante un período de actividades académicas, al consultar información en el SGA la página tardó más de lo normal en cargar y en algunos intentos tuve que actualizarla porque no mostraba la información. Esperé unos minutos, volví a intentar y el sistema respondió con normalidad. No puedo confirmar la causa; mi hipótesis es que muchos estudiantes accedían a la vez. Lo relaciono con **disponibilidad y tiempo de respuesta bajo demanda**: el servicio no estuvo caído, pero durante un lapso no respondió a tiempo. Frente a HDFS, la diferencia es que allí la falla (un nodo) quedó oculta por la replicación; aquí el problema, si fue de carga, afecta a todos los usuarios y la replicación de datos no lo evita.

5.2. WhatsApp: mensaje con un solo visto

A finales de octubre de 2026 envié un mensaje que permaneció un tiempo con un solo visto. No hice nada especial: esperé, revisé la conversación y después apareció como entregado y leído. No recuerdo cuánto tiempo pasó ni conozco la causa. Lo relaciono con la **comunicación asíncrona**: el emisor no necesita que el receptor procese el mensaje en el mismo instante, y el estado del mensaje se puso al día más tarde. Es parecido a los latidos de HDFS en que ambos dependen de mensajes que pueden retrasarse, pero a diferencia de HDFS aquí no hay un coordinador que me muestre con exactitud qué ocurrió.

5.3. Qué enseñan frente al experimento

En el experimento con HDFS comprobé que la caída de un DataNode no provocó la pérdida del archivo, porque sus bloques estaban replicados, y que los bloques volvieron a tener tres réplicas. Los dos ejemplos muestran problemas distintos: respuesta lenta bajo demanda y retraso en la comunicación. Los tres casos coinciden en que un sistema distribuido no puede asumir que todos sus componentes y comunicaciones funcionen perfectamente en todo momento.

6. Comparación con Ceph

Criterio	HDFS	Ceph
Ubicación de los datos	El NameNode guarda los metadatos y la ubicación de cada bloque [4]	Se calcula con CRUSH; clientes y OSD no dependen de una tabla central de consulta [3]; la elección de nodos se basa solo en la capacidad de almacenamiento, lo que puede causar desbalance de carga en sistemas heterogéneos [10]
Redundancia	Replicación de bloques (3 por defecto en mi montaje)	Replicación (típicamente 3 copias) o codificación por borrado [6]
Dominios de fallo	Conciencia de racks para repartir réplicas [4]	CRUSH reparte los objetos entre dominios de fallo: disco, host, rack, energía o switch [6]
Detección de fallos	Latidos de los DataNodes al NameNode; medí 14 min 02 s hasta declarar muerto un nodo	Cada OSD revisa su estado y el de otros y lo informa a los monitores [3]
Coordinación y consistencia	NameNode único como árbitro de metadatos; un solo escritor por archivo [4]	Monitores con Paxos y mayoría para el mapa del clúster [3]; CephFS busca consistencia como un sistema de archivos local [6]
Metadatos y escalabilidad	El NameNode no escala; limita el crecimiento [1]	Metadatos de CephFS atendidos por pocos servidores MDS reemplazables, que se reasignan para crecer [6]
Uso típico	Archivos grandes de escritura única	CephFS como reemplazo de NFS: directorios personales, almacenamiento temporal de HPC [6]

Cuadro 3: HDFS frente a Ceph en siete criterios.

Discusión. La diferencia central es cómo se encuentra un dato. HDFS pregunta a un servidor que lo recuerda; Ceph lo calcula. Eso explica la columna de escalabilidad: el NameNode concentra el conocimiento y por eso es el punto que limita el crecimiento [1], mientras que Ceph evita ese cuello de botella [3]. La contrapartida es que HDFS simplifica el diseño: un único escritor y un solo árbitro de metadatos hacen más fácil razonar sobre la consistencia, pero cobran su precio en concurrencia. Ceph, en cambio, ofrece un modelo más parecido a un sistema de archivos local, con varios clientes. Sobre el rendimiento en fallos, el estudio de Peters y van der Ster describe un caso en que un solo disco con mala conexión bajó el rendimiento del clúster de 25 GiB/s a menos de 5 GiB/s y Ceph no lo detectaba ni avisaba [6]; es un tipo de falla distinto (lentitud, no caída) al que probé en HDFS, y no puedo afirmar cómo respondería HDFS ante él.

7. Postura crítica

Dónde HDFS es una buena decisión. Para guardar archivos grandes que se escriben una vez y se leen muchas veces, la combinación de bloques grandes, replicación y lectura directa a los DataNodes es sencilla y robusta. Mi experimento lo respalda en el caso de un nodo caído: el archivo siguió disponible y la redundancia se restableció sin intervención.

Dónde no lo elegiría. Si necesitara que varios clientes escriban a la vez sobre los mismos archivos, como en un directorio compartido, HDFS no me serviría por su escritor único [4], y CephFS estaría más cerca de lo que necesito [6]. Y si esperara un crecimiento muy grande del número de archivos, el NameNode sería mi principal preocupación [1].

Lo que mi evidencia no demuestra. Probé un solo escenario (un DataNode, una vez, en un solo computador). No probé la caída del NameNode, que en mi montaje es el punto más débil; no medí la recuperación de varios fallos simultáneos; y el incidente de LinkedIn [9] muestra que la replicación no basta cuando fallan muchos nodos a la vez. Tampoco tengo evidencia propia sobre

Ceph: sus datos provienen de la documentación y de un artículo. La detección tardía (14 min) es aceptable para almacenamiento por lotes, pero sería un problema si una aplicación necesitara reaccionar rápido ante la caída de un nodo.

8. Conclusiones

1. En mi clúster, apagar un DataNode no hizo perder el archivo ni cambió el comando de lectura: tres bloques con tres réplicas bastaron para seguir leyendo con tres nodos.

2. La detección de la falla no es inmediata: el NameNode tardó 14 min 02 s en declarar muerto el nodo, y durante ese tiempo el informe seguía mostrando cuatro nodos vivos.

3. Tras la detección, los bloques volvieron a tener tres réplicas en los nodos restantes y `fsck` indicó estado `HEALTHY` con 0 bloques subreplicados.

4. Clasifico a HDFS como orientado a consistencia por su NameNode único y su escritor único; es un razonamiento mío apoyado en la documentación, no una clasificación publicada, y mi experimento no probó una partición de red.

5. La principal diferencia con Ceph es que Ceph calcula la ubicación con CRUSH y HDFS la consulta al NameNode; por eso el NameNode limita la escalabilidad de HDFS.

6. Los dos ejemplos de mi entorno (lentitud del SGA y retraso en el estado de un mensaje de WhatsApp) muestran que los problemas de disponibilidad y comunicación aparecen también en sistemas que uso a diario, aunque no pude determinar sus causas.

7. Los límites de mi trabajo son una sola repetición, discos compartidos, ausencia de prueba del NameNode y ausencia de experimento propio con Ceph.

9. Declaración de uso de inteligencia artificial

Herramienta y versión: Claude, de Anthropic, usado en una sesión de Claude Code configurada con el modelo `claude-sonnet-5-5` (el modelo que atiende cada respuesta puede variar durante una sesión).

Para qué lo usé: (1) que me explicara los conceptos de HDFS y del experimento; (2) elegir la estructura del repositorio y del informe; (3) preparar la configuración de Docker y el guion de comandos del experimento; (4) buscar fuentes candidatas; (5) redactar un borrador del informe en LaTeX y dibujar en TikZ el diagrama de la figura 1 a partir de mi boceto a mano; (6) revisar mi interpretación de resultados y señalarme afirmaciones que mi evidencia no respaldaba.

Cómo verifiqué: el boceto a mano (Anexo A) lo dibujé yo. Ejecuté yo mismo cada comando del experimento y las capturas son de mi computador con mi nombre de usuario, el nombre de la máquina y la hora visibles. Abrí cada DOI de las referencias y comparé autores, año, título, revista, volumen y páginas con el `.bib`; las tres capturas de lectura anotadas del Anexo B corresponden a pasajes que leí en los PDF originales. Revisé el diagrama contra mi boceto y contra el `fsck` real. Los ejemplos de la sección 5 son situaciones que viví.

Referencias

[1] P. M. Dhulavvagol y S. G. Totad, «Performance Enhancement of Distributed System Using HDFS Federation and Sharding,» *Procedia Computer Science*, vol. 218, págs. 2830-2841, 2023. DOI: 10.1016/j.procs.2023.01.254

[2] K. V. Shvachko, C. Liang y S. Dzinamarira. «The exabyte club: LinkedIn's journey of scaling the Hadoop Distributed File System.» Publicado el 27 de mayo de 2021, LinkedIn Engineering, visitado 13 de nov. de 2026. dirección: https://engineering.linkedin.com/blog/2021/the-exabyte-club--linkedin-s-journey-of-scaling-the-hadoop-distr

[3] Ceph Foundation. «Architecture — Ceph Documentation (release Reef),» visitado 13 de nov. de 2026. dirección: https://docs.ceph.com/en/reef/architecture/

[4] Apache Software Foundation. «HDFS Architecture (Apache Hadoop 3.3.0).» Documentación de la versión 3.3.0, publicada en 2020, visitado 13 de nov. de 2026. dirección: https://hadoop.apache.org/docs/r3.3.0/hadoop-project-dist/hadoop-hdfs/HdfsDesign

[5] J. Ahmed, A. Karpenko, O. Tarasyuk, A. Gorbenko y A. Sheikh-Akbari, «Consistency Issue and Related Trade-offs in Distributed Replicated Systems and Databases: A Review,» *Radioelectronic and Computer Systems*, n.º 2, págs. 171-179, 2023. DOI: 10.32620/reks.2023.2.14

[6] A. J. Peters y D. C. van der Ster, «Evaluating CephFS Performance vs. Cost on High-Density Commodity Disk Servers,» *Computing and Software for Big Science*, vol. 5, 2021, Art. n.º 25. DOI: 10.1007/s41781-021-00071-1

[7] N. Alange y P. V. Sagar, «Small files access efficiency in hadoop distributed file system a case study performed on British Library text files,» *Cluster Computing*, vol. 26, n.º 6, págs. 3381-3388, 2023. DOI: 10.1007/s10586-023-03992-1

[8] M. Saadoon, S. H. A. Hamid, H. Sofian, H. H. M. Altarturi, Z. H. Azizul y N. Nasuha, «Fault tolerance in big data storage and processing systems: A review on challenges and solutions,» *Ain Shams Engineering Journal*, vol. 13, n.º 2, 2022, Art. n.º 101538. DOI: 10.1016/j.asej.2021.06.024

[9] S. Ramu y V. Rajamani. «Theory vs. Practice: Learnings from a recent Hadoop incident.» Publicado el 6 de agosto de 2020, LinkedIn Engineering, visitado 13 de nov. de 2026. dirección: https://engineering.linkedin.com/blog/2020/learnings-from-a-recent-hadoop-incident

[10] Z. Li e Y. Wang, «An adaptive read/write optimized algorithm for Ceph heterogeneous systems via performance prediction and multi-attribute decision making,» *Cluster Computing*, vol. 26, n.º 2, págs. 1125-1146, 2023. DOI: 10.1007/s10586-022-03764-3

A. Boceto a mano de la arquitectura

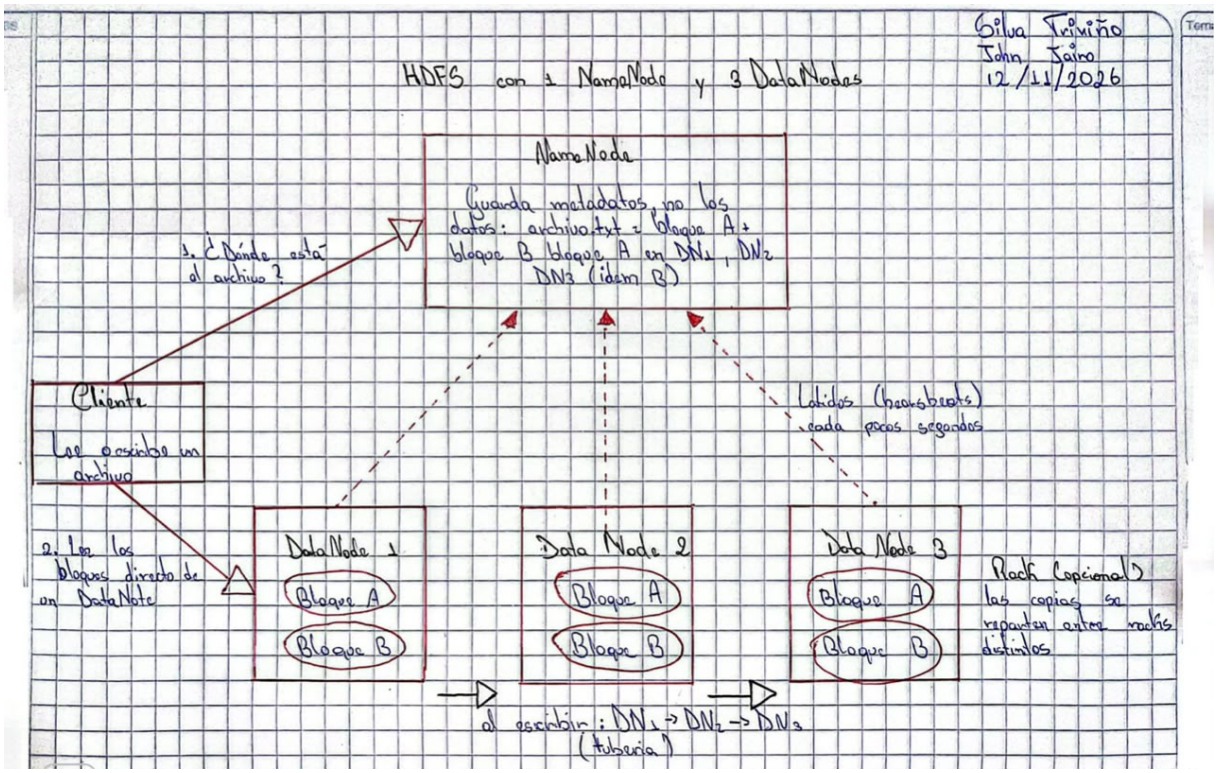


Figura 6: Boceto a mano de HDFS con 1 NameNode y 3 DataNodes (Silva Triviño John Jairo, 12/11/2026), punto de partida del diagrama de la figura 1.

B. Capturas de lectura de las fuentes

Cada captura corresponde a una cita del informe; la página es la que se ve en el PDF.

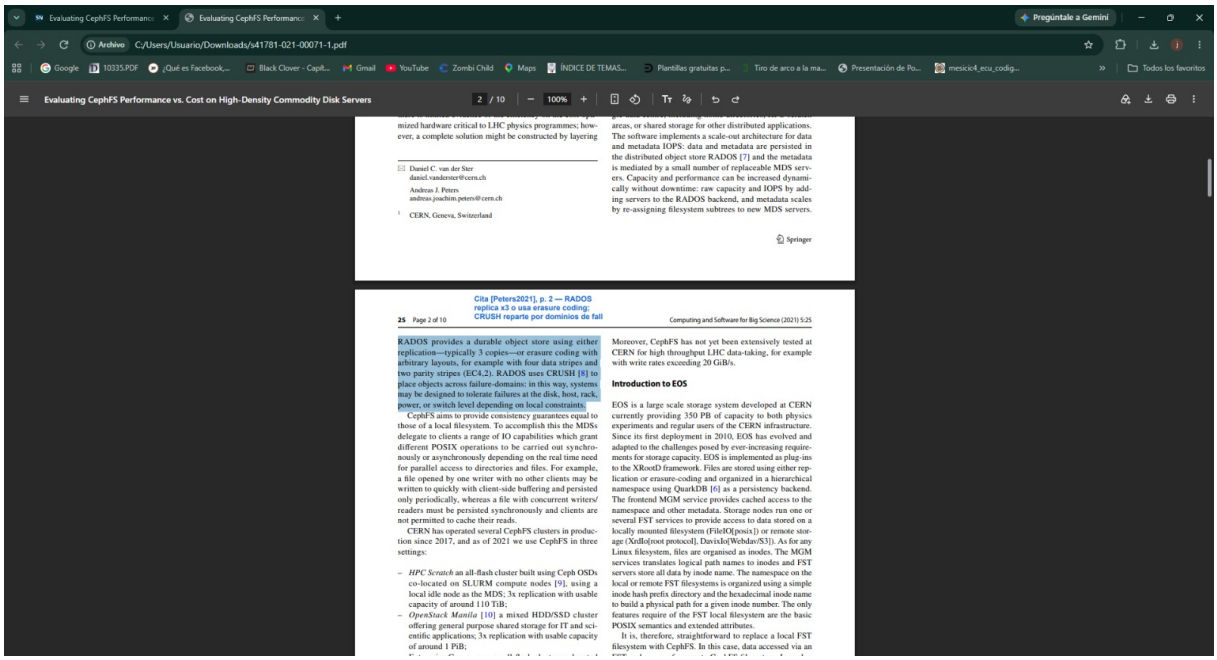


Figura 7: Peters y van der Ster [6], p. 2: replicación o codificación por borrado, y CRUSH reparte por dominios de fallo.



Ain Shams Engineering Journal
Volume 13, Issue 2, March 2022, 101538



Fault tolerance in big data storage and processing systems: A review on challenges and solutions

Muntadher Saadoon ✉, Siti Hafizah Ab. Hamid ✉, Hazrina Sofian ✉, Hamza H.M. Altarturi ✉, Zati Hakim Azizul ✉, Nur Nasuha ✉

Show more

+ Add to Mendeley Share Cite

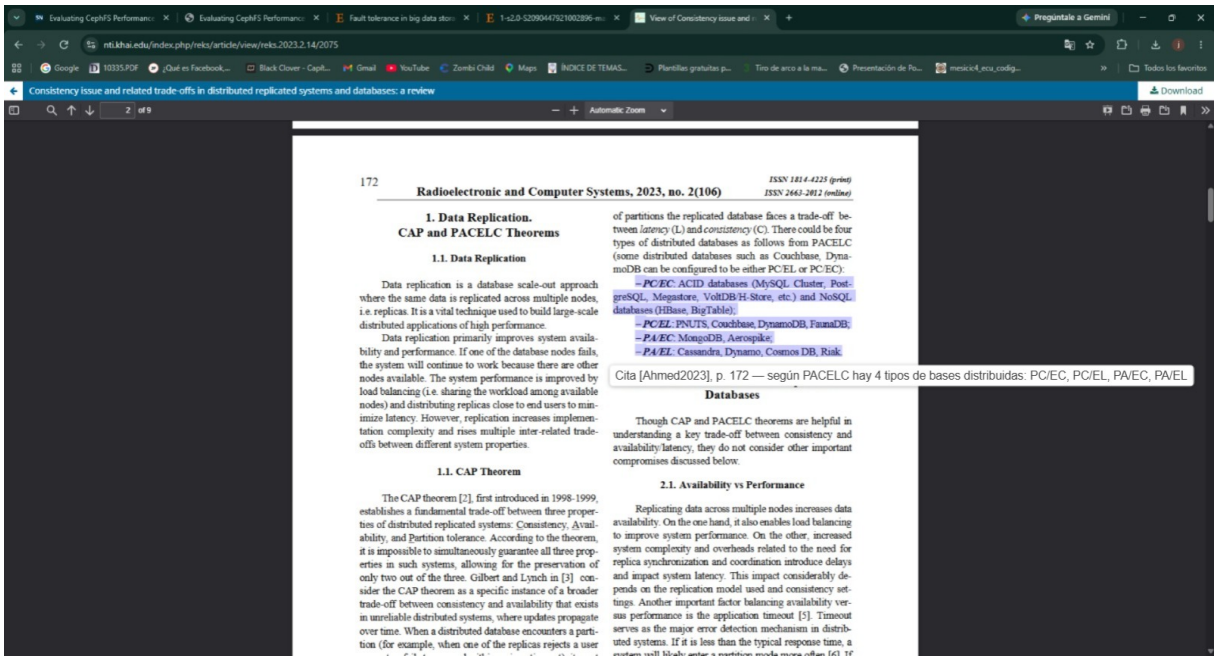
https://doi.org/10.1016/j.asej.2021.06.024 ↗ Get rights and content ↗ View accessibility information

Under a Creative Commons license ↗ Open access

Abstract

Big data systems are sufficiently stable to store and process a massive volume of rapidly changing data. However, big data systems are composed of large-scale hardware resources that make their subspecies easily fail. Fault tolerance is the main property of such systems because it maintains availability, reliability, and constant performance during faults. Achieving an efficient fault tolerance solution in a big data system is challenging because fault tolerance must meet some constraints related to the system performance and resource consumption. This study aims to provide a consistent understanding of fault tolerance in big data systems and highlights common challenges that hinder the improvement in fault tolerance efficiency. The fault tolerance solutions applied by previous studies intended to address the identified challenges are reviewed. The paper also presents a perceptive discussion of the findings derived from previous studies and proposes a list of future directions to address the fault tolerance challenges.

Figura 8: Saadoon et al. [8], p. 3 del PDF: heartbeats y detección de fallos, usado en HDFS.



172 Radioelectronic and Computer Systems, 2023, no. 2(106)

ISSN 1814-4225 (print)
ISSN 2663-2012 (online)

1. Data Replication. CAP and PACELC Theorems

1.1. Data Replication

Data replication is a database scale-out approach where the same data is replicated across multiple nodes, i.e. replicas. It is a vital technique used to build large-scale distributed applications of high performance.

Data replication primarily improves system availability and performance. If one of the database nodes fails, the system will continue to work because there are other nodes available. The system performance is improved by load balancing (i.e. sharing the workload among available nodes) and distributing replicas close to end users to minimize latency. However, replication increases implementation complexity and rises multiple inter-related trade-offs between different system properties.

1.1. CAP Theorem

The CAP theorem [2], first introduced in 1998-1999, establishes a fundamental trade-off between three properties of distributed replicated systems: Consistency, Availability, and Partition tolerance. According to the theorem, it is impossible to simultaneously guarantee all three properties in such systems, allowing for the preservation of only two out of the three. Gilbert and Lynch in [3] consider the CAP theorem as a specific instance of a broader trade-off between consistency and availability that exists in unreliable distributed systems, where updates propagate over time. When a distributed database encounters a partition (for example, when one of the replicas rejects a user

of partitions the replicated database faces a trade-off between *latency* (L) and *consistency* (C). There could be four types of distributed databases as follows from PACELC (some distributed databases such as Couchbase, DynamoDB can be configured to be either PC/EL or PC/EC):

– ***PC/EC***: ACID databases (MySQL Cluster, PostgreSQL, Megastore, VoltDB/H-Store, etc.) and NoSQL databases (HBase, BigTable);
– ***PC/EL***: PNUTS, Couchbase, DynamoDB, FaunaDB;
– ***PA/EC***: MongoDB, Aerospike;
– ***PA/EL***: Cassandra, Dynamo, Cosmos DB, Riak.

Databases

Though CAP and PACELC theorems are helpful in understanding a key trade-off between consistency and availability/latency, they do not consider other important compromises discussed below.

2.1. Availability vs Performance

Replicating data across multiple nodes increases data availability. On the one hand, it also enables load balancing to improve system performance. On the other, increased system complexity and overheads related to the need for replica synchronization and coordination introduce delays and impact system latency. This impact considerably depends on the replication model used and consistency settings. Another important factor balancing availability versus performance is the application timeout [5]. Timeout serves as the major error detection mechanism in distributed systems. If it is less than the typical response time, a

Figura 9: Ahmed et al. [5], p. 172: los cuatro tipos de PACELC.